Name of Student : Raghav Mahesh Sarda	**PRN:** 25070122***235***
Subject: Programming with Java	**Semester: III**
ASSIGNMENT No : 10	**Year: AY 2026-27**
TITLE : Abstract Classes & Methods	

Problem Statement

Write a Java program to demonstrate the use of abstract classes and abstract methods.

Learning Objectives

To implement abstraction using abstract classes and abstract methods.

Program:

PaymentSystem.java

```
// Assignment 10 - Exercise 1: Abstract Payment Class
import java.util.*;
abstract class Payment // Abstract class
{
   abstract void processPayment(double amt); // Abstract method
}

class CreditCard extends Payment // Child Class 1
{
   String cardNo;
   CreditCard(String c) // Constructor
   {
       cardNo = c;
   }
   void processPayment(double amt) // Implementing abstract method
   {
       System.out.println("Processing Credit Card payment of $" + amt +
" for Card No: " + cardNo);
   }
}
class UPI extends Payment // Child Class 2
{
```

```
    String upiId;
    UPI(String u) // Constructor
    {
        upiId = u;
    }
    void processPayment(double amt) // Implementing abstract method
    {
        System.out.println("Processing UPI payment of $" + amt + " for
UPI ID: " + upiId);
    }
}

public class PaymentSystem
{
    public static void main(String args[]) // Main Method
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Credit Card Number: ");
        String c = sc.next();
        System.out.print("Enter CC Payment Amount: ");
        double camt = sc.nextDouble();
        System.out.print("Enter UPI ID: ");
        String u = sc.next();
        System.out.print("Enter UPI Payment Amount: ");
        double uamt = sc.nextDouble();

        Payment p1 = new CreditCard(c); // Upcasting
        Payment p2 = new UPI(u); // Upcasting

        System.out.println("\n-- Transaction Status --");
        p1.processPayment(camt); // Calling overridden CC method
        p2.processPayment(uamt); // Calling overridden UPI method
        sc.close();
    }
}
```

FoodOrderSystem.java

```
// Assignment 10 - Exercise 2: Abstract FoodOrder Class
import java.util.*;
abstract class FoodOrder // Abstract class
{
   double baseAmount;
   FoodOrder(double amt) // Constructor for abstract class
   {
       baseAmount = amt;
   }
   abstract void calculateBill(); // Abstract method
}

class DineInOrder extends FoodOrder // Child Class 1
{
   double serviceCharge;
   DineInOrder(double amt, double sc) // Constructor
   {
       super(amt); // Initializing parent property
       serviceCharge = sc;
   }
   void calculateBill() // Implementing abstract method
   {
       double total = baseAmount + serviceCharge;
       System.out.println("Dine-In Total Bill: $" + total + " (Includes
Service Charge: $" + serviceCharge + ")");
   }
}

class TakeAwayOrder extends FoodOrder // Child Class 2
{
   double packingCharge;
   TakeAwayOrder(double amt, double pc) // Constructor
   {
       super(amt); // Initializing parent property
       packingCharge = pc;
   }
   void calculateBill() // Implementing abstract method
   {
       double total = baseAmount + packingCharge;
       System.out.println("Take-Away Total Bill: $" + total + "
(Includes Packing Charge: $" + packingCharge + ")");
   }
}
```

```java
public class FoodOrderSystem
{
   public static void main(String args[]) // Main Method
   {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Dine-In Base Amount: ");
        double da = sc.nextDouble();
        System.out.print("Enter Service Charge: ");
        double scg = sc.nextDouble();
        System.out.print("\nEnter Take-Away Base Amount: ");
        double ta = sc.nextDouble();
        System.out.print("Enter Packing Charge: ");
        double pcg = sc.nextDouble();

        FoodOrder f1 = new DineInOrder(da, scg); // Instantiating
DineInOrder
        FoodOrder f2 = new TakeAwayOrder(ta, pcg); // Instantiating
TakeAwayOrder

        System.out.println("\n-- Order Bills --");
        f1.calculateBill(); // Executing DineIn bill calculation
        f2.calculateBill(); // Executing TakeAway bill calculation
        sc.close();
   }
}
```

Output:

PaymentSystem.java

```
Problems 10  Output  Debug Console  Terminal  Ports

cd "/Users/thegharkalaptop/Documents/Programming (The RS)/" && javac PaymentSystem.java && java PaymentSystem
thegharkalaptop@The-Ghar-Ka-Laptop Programming (The RS) % cd "/Users/thegharkalaptop/Documents/Programming (The RS)/" && javac PaymentSystem.java && java PaymentSystem
Enter Credit Card Number: 1234-5678-9012-3456
Enter CC Payment Amount: 200
Enter UPI ID: raghav@okswiss
Enter UPI Payment Amount: 100

-- Transaction Status --
Processing Credit Card payment of $200.0 for Card No: 1234-5678-9012-3456
Processing UPI payment of $100.0 for UPI ID: raghav@okswiss
thegharkalaptop@The-Ghar-Ka-Laptop Programming (The RS) %
```

FoodOrderSystem.java



Conclusion:

In this assignment, data abstraction was achieved using abstract classes and methods. By defining a partial blueprint in the parent class without implementation details, the child classes were strictly forced to implement their own specific logic for dynamic tasks like payment processing and billing. This demonstrated the power of abstraction in hiding internal complexities and providing clean, modular object-oriented code.